

Notes on C++ Package Management

- Document number: P1254R0
- Date: 2018-10-07
- Author: Steve Downey <sdowney2@bloomberg.net>
- Audience: SG15

Abstract: Text interfaces should be defined to be used by package management, package build, and build systems to allow more effective C++ package management for users by allowing package management tools to drive the build of source packages without unhelpful help from the package's build system.

Table of Contents

- [1. Introduction](#)
- [2. Definitions](#)
- [3. Assumptions](#)
- [4. Package DAG and Cycles](#)
- [5. Install Layout](#)
- [6. Package build system requirements](#)
- [7. Summary](#)
- [8. References](#)

1 Introduction

C++ is notable among modern languages in not providing a package management system by which developers can easily install and reuse code developed by others into their own projects. It shares this with C, and for similar reasons. Until recently, source distribution was the exception, rather than the rule, and pre-compiled libraries are fragile, as they must, in general, use the exact same set of dependent libraries, compiled the same way. This largely limits reuse of packages to those provided by an OS or distro vendor. A modern Linux or BSD system may provide 1000s of such packages, however changing any of them may cause the entire system to fail. There is huge tension between Long Term Support versions and making current packages available.

Mature organizations will often maintain an entire parallel universe of packages, independent of the vendor provided libraries, or even becoming an OS vendor themselves, to themselves. This can be very expensive, as there is little to no standardization of tooling around package maintenance, and what tooling there is tends to be language agnostic, which means poor support for toolchain integration.

Languages with few or one implementation can bless a single build tool and package manager. That is not the case for C++, and is unlikely to be the case any time soon. However, it should be possible to define interfaces between build tools and package managers so they can work in concert and still provide good user experience.

There is existing practice that can be leveraged in pkg-config and Debian dpkg source package control files, however there is room for improvement in describing C++ specific requirements.

2 Definitions

"When I use a word," Humpty Dumpty said, in a rather scornful tone, "it means just what I choose it to mean - neither more nor less."

Binary Package: A binary software component and the information necessary to use it. Libraries and header files, the meta-data to include it into a project, and the packages this package depends on.

Package Manager: A system that makes a consistent set of binary packages available to be included in a project. The package manager will ensure that updating one binary package entails updating all of the packages that use it, and ensuring that dependent packages are also installed. A tool like apt is an example of a package manager.

Source Package: The sources used to build a binary package, meta-data about the dependencies, and a scriptable interface the package builder can use to drive the build against other packages and with a particular toolchain.

Toolchain: The compilers, linker, system libraries, standard C++ library, C runtime, used to convert source to binary. Objects built with the same toolchain are minimally compatible. They are, for example, all 64 bit using c++17 against the non-debug version of the standard library. Packages still might not be compatible if they were not built against the same set of dependencies.

Package Builder: Converts a source package to a package. It will drive the build with a particular toolchain communicating what dependent packages should be used to build against. Tools like sbuild and pbuilder are package builders.

Distro: A consistent set of binary packages and the source packages that produce them that can be used with each other. There is only one version of a binary package available from a distro.

Distro Manager: When a source package is promoted into a distro, it will manage the rebuild of all source packages by the package builder of any package that depends on the package being promoted. It will gate promotion, preventing incompatible packages from being promoted for use. HEAD may be the only version available. It may have facilities to deal with co-dependent packages that do not form a DAG.

Binary Integration: The results of building a package are made available to the development build. Public headers and suitable libraries are installed where the build can use them. The source may be available, but the development build will generally ignore changes made there. This is as opposed to Source Integration

Source Integration: The sources for the package are integrated into the development builds system. Changes made to the package sources will flow automatically through the build. There is no 'install' step. There are occasional hybrid systems where the package sources are built by the development system and then installed for the rest of the build to pick up. This usually produces headaches for the developer.

3 Assumptions

C++ does not, at scale, allow different versions of a package within an executable binary, although there are techniques to allow this in tightly controlled situations.

At scale, mixing C++ standards does not work. '11 and '14 are the exception, not the rule. Feature test macros mean detectable differences at compile time, leading to ODR violations.

There will not be a 'universal' anything.

There is not and will not be a standard build system. The existing diversity is too large.

Compiler flags are about as standardized as they are going to be.

Specifying compiler flags, beyond where to find dependent packages, is a decision for the user defining the toolchain. Although the case where a particular library needs unsafe or expensive optimizations needs some consideration.

Failure is an option. If a package can't work in the environment specified, it should fail, not attempt to provide a better environment for itself.

Although there will be more C++ code and packages written in the future than exists today, we have to deal with the transition. The build interfaces need to be adaptable and wrappable. We can not require a package to adopt an entirely new build system. We may require updating the build system, e.g. a new version of cmake, b2, etc.

For purposes of packaging, meta build systems **are** build systems. That cmake produces makefiles is an implementation detail.

4 Package DAG and Cycles

In a pre-module world, the build of packages can be done against all other packages to be used in parallel by installing the headers where every package can see them. While for testing purposes, it is useful to be able to arrange the packages in a DAG and test them from bottom to top, it is not strictly necessary. With separate compilation of translation units it suffices to make the headers of the co-dependent packages visible to each other and build each translation unit independently. This requires the ability to install headers from a source package. The resultant libraries may have cycles, but there are existing techniques for dealing with those.

C++ Modules, as last proposed, require a DAG between all modules. Automating this process is an open issue. However if package meta-data specifies module requirements in terms of the dependent packages, it will be straightforward to build each package and make the results, including binary module interfaces, available to dependent packages.

5 Install Layout

The layout of a package on Unix style systems is highly stereotyped. Headers go into an include directory, libraries go into a lib directory. There might be some architecture info in the name of the lib dir, such as lib64 for a 64bit library on a system that allows both 32 and 64 bit packages.

For system style packaging, all packages are installed into the same root, however there may be multiple roots, such as /usr/local, /usr, and /. This allows some flexibility for the admin to shadow particular packages when it is, for example, infeasible to upgrade a package in /usr used by the OS as a whole.

Opt style packaging puts a similar layout within the directory, but each package, or organization, has its own, unshared, directory. This allows versions in parallel, but the consumption of the packages is more complicated, particularly if there are interdependencies.

Packages on Windows often use an 'opt' style layout, however the conventions and norms are weaker. Packages are also much more likely to contain binaries for multiple architectures and toolchain options. Providing a library that uses the ABI incompatible debug version of the debug library is common and expected.

An installed package should provide metadata about how the package should be consumed. On Linux systems this is often a pkg-config, or pc, file. This provides compilation flags and linker flags to use the library. This is just barely adequate within a distro. Often the flags provided can have unintended consequences to consumers, for example providing an ABI affecting compiler define, language standard setting, or feature flag.

A more restricted form of package metadata needs to be developed, remembering that failure is an option. For example, if a particular -D must be provided, then the toolchain, which will be used across all packages, must provide it, rather than injecting it into the build command where it can break consuming packages.

6 Package build system requirements

The core package distribution mechanism will be source packages. At scale, no one else will be building with the exact toolchain and dependency versions as anyone else. It is unlikely that compute resources would be made available for free to produce binary packages on demand. Building binary packages will happen within an organization. It is possible that the binary packages produced can be shared within the organization, as upgrades of third party and second party packages are infrequent. However the model of composing a distro of binary packages can be scaled down to local development. This is essentially the virtual environment model of python.

There needs to be a treaty and demarc between the package build system and the build system of a package. An example of this is the `debian/rules` file used by the Debian package system which has a few well documented targets that the `dpkg` build system can invoke to produce binary package artifacts from source. This indirection also allows third party packaging of libraries by experts. It is not uncommon, even if the upstream library provides package information, for vendors to ignore the upstream.

Many build systems are quite helpful, and will find a required dependent package somewhere, or add necessary flags to the build, and in general try their best to build the software in some manner. This is a disaster for package consumers, and leads to gross or subtle ODR violations. Gross violations fail downstream builds. Subtle ones fail at runtime.

In the mode of being built under a package manager build, the build system must fail if it is disappointed by the toolchain or available packages. It must build against the packages the build manager tells it to. It must use the toolchain unaltered in any observable way that the build manager gives it. The common case failure mode if it does "help" is changing the meaning of the headers of a package it is using, leading to undefined behavior.

A source package must also be able to declare what its requirements are. The list of packages it needs to build, as well as the packages it needs available to run, the C++ features it needs, or the standard level it needs. This will allow the builder and package manager to detect problems early and communicate failures in a way that users will understand. Experience shows that compilation failures in the package are never easily comprehensible.

One way of expressing C++ language requirements might use the standard feature test macros. It would be straightforward for the package builder to test the requirements with a generated litmus test, without asking the package to test within its build system. There is long experience with this approach in autotools. Something like

```
Cpp Requires:
cplusplus >= 201703L
cpp_structured_bindings
cpp_lib_concepts >= 201806
```

which might generate code

```
static_assert(__cplusplus >= 201703L, "cplusplus >= 201703L");

#if !defined(__cpp_structured_bindings)
static_assert(false, "cpp_structured_bindings not defined");
#endif

#if !defined(__cpp_lib_concepts) && !(__cpp_lib_concepts >= 201806)
static_assert(false, "not cpp_lib_concepts >= 201806");
#endif
```

Solving version requirements, e.g. `libc6 (>= 2.0.105)`, is an NP-hard problem, demonstrated to be equivalent to SAT-3. In a model where there are different versions of source packages available, the package manager must solve for a single set and then build those together. Other languages package managers, such as `npm`, avoid this

by making multiple versions available at runtime. This is not feasible for C++. This also implies a source package model, as experience has demonstrated that for C++ binary packages only work with the packages they were built against. Maintaining library ABI is fragile, and many projects aim for source compatibility only. That is, they expect existing consumers to successfully recompile, but recompilation will be necessary. The distro model assumes a curated set of packages, and largely side-steps the version management problem.

7 Summary

There is a clear desire for package management in the C++ community. And envy when a C++ developer works with, Rust, Python, Haskell, or any number of other languages. Open Source distribution is now normal, and makes package management possible, as it allows packages to be rebuilt when underlying dependent packages change, a requirement for C++. However, it is still too difficult to integrate packages with dependencies of more than the standard library into a project. Some build systems and meta build systems provide some facilities for source integration into a project, which helps reuse, but requires build system standardization, which is not going to happen. For example, for GTest, the typical and recommended practice is to include the gtest's cmake build into the consuming project's build, while excluding it from all. This has the unfortunate side-effect of every project having a slightly different version of GTest, and each project having to repeat the build.

Standardizing the interfaces for consuming and producing binary packages will allow greater reuse of code. Most application build systems are already well suited to consume binary packages. Build systems for source packages may require some work to allow the control necessary, but not entire rework or replacement. There are well established practices in the various Linux, BSD, and MacOS package management systems that can be either adapted and expanded, or used as a model, to provide C++ specific modern package management.

8 References

Pkg-config: <https://www.freedesktop.org/wiki/Software/pkg-config/>

Debian package information: <https://www.debian.org/doc/debian-policy/index.html>

- in particular <https://www.debian.org/doc/debian-policy/ch-controlfields.html>
- and <https://www.debian.org/doc/debian-policy/ch-relationships.html>

Date: 2018-10-07 Sun 00:00

Author: Steve Downey

Created: 2018-10-08 Mon 09:42

[Validate](#)